

UNIT 3

SHORT ANSWER QUESTIONS

~~Q~~. Explain the BETWEEN operator with suitable example.

The BETWEEN operator is used to check whether a value lies within a specified range, including boundary values.

Example:

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));
insert into Student1 values(1,"Deepu",19,"CSE"),
(2,"Nani",21,"ECE"),
(3,"Rani",20,"ECE"),
(4,"Joseph",17,"CSE"),
(5,"Madhu",18,"CSE");
select * from Student1;
select * from Student1 where age between 18 and 20;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
3	Rani	20	ECE
5	Madhu	18	CSE

This query returns students whose age is between 18 and 20.

~~Q~~. Discuss the Operator Precedence in SQL.

Operator precedence in SQL determines the order in which operators are evaluated in a query when multiple operators are used together. Operators with higher precedence are evaluated first.

Order of Operator Precedence

1. Parentheses ()- Force explicit grouping
2. Unary operators: +, - (bitwise NOT)
3. Multiplication *, Division /, Modulo %
4. Addition +, Subtraction -, Concatenation ||
5. Comparison operators
6. Logical Operators
7. Assignment = (in SET clauses or variable assignment)

3Q.Explain The IN operator in SQL with suitable Example.

IN Operator:

The IN operator is used to check whether a value matches any value in a specified list.

Example:

```
select * from Student1 where age in (18,20) ;
```

Output:

id	name	age	dept
3	Rani	20	ECE
5	Madhu	18	CSE

This query returns students whose age is in 18 or 20.

4Q.What are SQL aggregate functions? Name any two.

SQL aggregation function is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.

o It is also used to summarize the data.

Types of aggregate functions:

- ☐ Count()
- ☐ Sum()
- ☐ Avg()
- ☐ Min()
- ☐ Max()

5Q.What are LIMIT and OFFSET clauses in SQL?

LIMIT and OFFSET in SQL:

LIMIT and OFFSET clauses are used to control the number of rows returned in a query result.

LIMIT specifies the maximum number of rows to display.

OFFSET specifies the number of rows to skip before displaying results.

id	name	marks
1	Ravi	85
2	Anu	90
3	Kiran	78
4	Sita	88
5	Ramesh	92

Query:

```
SELECT * FROM students
```

```
LIMIT 2 OFFSET 2;
```

Output:

id	name	marks
3	Kiran	78
4	Sita	88

The query skips the first 2 rows and returns the next 2 rows from the table.

6Q. Define a view in SQL and write the syntax to create a view.

Views in SQL

- Views in SQL are considered as a virtual table. A view also contains rows and columns.
- To create the view, we can select the fields from one or more tables present in the database.

A view can either have specific rows based on certain condition or all the rows of a table.

Sample table:

```
CREATE VIEW
```

Syntax:

```
CREATE VIEW <view_name> AS SELECT column1, column2.. column n  
FROM <table_name>
```

LONG ANSWER QUESTIONS

~~Q.~~ Logical Operators with Definitions and Examples

Logical Operators:

Logical operators are used in SQL WHERE clauses to combine multiple conditions. They return TRUE or FALSE depending on the logic.

The logical operators are

1. AND Operator
2. OR Operator
3. NOT Operator

To understand the logical operators, first create a table with id, name, age, dept columns as shown in below.

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));
insert into Student1 values(1,"Deepu",19,"CSE"),
(2,"Nani",21,"ECE"),
(3,"Rani",20,"ECE"),
(4,"Joseph",17,"CSE"),
(5,"Madhu",18,"CSE");
select * from Student1;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

a) AND Operator

Definition:

The AND operator is used to combine two or more conditions and returns results only when all conditions are true

Syntax:

```
SELECT * FROM Tablename
```

```
WHERE Condition1 AND Condition2;
```

Example:

```
select * from Student1 where age>18 AND dept="CSE";
```

Output:

	id	name	age	dept
	1	Deepu	19	CSE

The query returns the values whose age is greater than 19 and belongs to CSE

b) OR Operator

Definition:

The OR operator is used to combine multiple conditions and returns results when at least one condition is true.

Syntax:

```
SELECT * FROM Table_name
```

```
WHERE Condition1 OR Condition2;
```

Example:

```
select * from Student1 where age>20 OR dept="CSE";
```

Output:

	id	name	age	dept
	1	Deepu	19	CSE
	2	Nani	21	ECE
	4	Joseph	17	CSE
	5	Madhu	18	CSE

This query returns students whose age is greater than 20 or belong to CSE .

c)NOT Operator**Definition:**

The NOT operator is used to reverse a condition and returns records that do not satisfy the given condition.

Stntax:

SELECT * FROM Table_name

WHERE NOT Condition;

Example:

```
select * from Student1 where NOT dept="CSE";
```

Output:

	id	name	age	dept
	2	Nani	21	ECE
	3	Rani	20	ECE

This query returns students who do not belong to the CSE course.

~~2~~Q.Explain Comparison operators in DBMS with Examples

Comparison operators are used to compare values in SQL queries and filter data.

Types of Comparison Operators

There are many types of comparison operators are there.they are

1. Equal (=)
2. Not Equal (!= or <>)
3. Greater Than (>)
4. Less Than (<)
5. Greater Than or Equal To (>=)
6. Less Than or Equal To (<=)

To understand the Comparison operators,first create a table with id,name,age,dept columns as shown in below.

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));
insert into Student1 values(1,"Deepu",19,"CSE"),
(2,"Nani",21,"ECE"),
(3,"Rani",20,"ECE"),
(4,"Joseph",17,"CSE"),
(5,"Madhu",18,"CSE");
select * from Student1;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

1. Equal (=)

It Checks whether values are equal.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name = value;
```

Example:

```
select * from Student1 where dept="ECE";
```

Output:

id	name	age	dept
2	Nani	21	ECE
3	Rani	20	ECE

The above query returns whose dept is ECE as shown in above output

2. Not Equal (!= or <>)

It Checks whether values are not equal.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name != value;
```

Example:

```
select * from Student1 where age!=20;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

The above query returns the table by removing the row whose age is 20 as shown in above output

3. Greater Than (>)

It Returns values greater than specified value.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name > value;
```

Example:

```
select * from Student1 where age>19;
```

Output:

id	name	age	dept
2	Nani	21	ECE
3	Rani	20	ECE

The above query returns whose age is greater than 19 as shown in above output

4. Less Than (<)

It Returns values less than specified value.

Syntax:

```
SELECT * FROM Table_name  
WHERE Column_name <value;
```

Example:

```
select * from Student1 where age<19;
```

Output:

id	name	age	dept
4	Joseph	17	CSE
5	Madhu	18	CSE

The above query returns whose age is less than 19 as shown in above output

4. Greater Than or Equal To (>=)

It Returns values greater than or equal to specified value.

Syntax:

```
SELECT * FROM Table_name
WHERE Column_name >= value;
```

Example:

```
select * from Student1 where age>=19;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE

The above query returns whose age is greater than 19 as shown in above output

5. Less Than or Equal To (<=)

It Returns values less than or equal to specified value.

Syntax:

```
SELECT * FROM Table_name
WHERE Column_name <= value;
```

Example:

```
select * from Student1 where age<=19;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
4	Joseph	17	CSE
5	Madhu	18	CSE

The above query returns whose age is less than 19 as shown in above output

~~3Q~~. Explain Pattern Matching using LIKE operator in DBMS.

The SQL LIKE operator is used to search for a specific pattern within a column's text data. It works with wildcard characters to match partial strings, making it useful for flexible filtering.

- % represents any sequence of characters

To understand the Pattern matching using LIKE operators, first create a table with id, name, age, dept columns as shown in below.

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));  
insert into Student1 values(1,"Deepu",19,"CSE"),  
(2,"Nani",21,"ECE"),  
(3,"Rani",20,"ECE"),  
(4,"Joseph",17,"CSE"),  
(5,"Madhu",18,"CSE");  
select * from Student1;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

Syntax

```
SELECT column_name  
FROM table_name  
WHERE column_name LIKE pattern;
```

Example 1:

In this we write a query to return whose name is end with “u”.For that we use the syntax “%u” as shown in below example.

```
select * from Student1 where name like "%u";
```

Output:

id	name	age	dept
1	Deepu	19	CSE
5	Madhu	18	CSE

In above output we observe that the two names **Deepu** and **Madhu** are end with “u”

Example 2:

In this we write a query to return whose name is starts with “n”.For that we use the syntax “%n” as shown in below example.

```
select * from Student1 where name like "n%";
```

Output:

id	name	age	dept
2	Nani	21	ECE

4. What are SQL functions in detail and write about String functions.

SQL Functions:

SQL Functions are predefined commands used to perform operations on data and return a result.

Key Advantages:

Reusability: Functions allow you to write logic once and invoke it multiple times, reducing redundant code in queries or stored procedures.

Performance Boost: Built-in or user-defined functions execute faster than inline calculations, especially for repetitive tasks like aggregations or string manipulations.

Modularity: They encapsulate business logic, making code easier to maintain, debug, and test independently.

Readability: Queries become cleaner and more readable by replacing verbose expressions with concise function calls.

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));
insert into Student1 values(1,"Deepu",19,"CSE"),
(2,"Nani",21,"ECE"),
(3,"Rani",20,"ECE"),
(4,"Joseph",17,"CSE"),
(5,"Madhu",18,"CSE");
select * from Student1;
```

Output:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

1. String Functions

i)UPPER()

It Convert text of column to uppercase.

Syntax:

UPPER(column_name)

Example:

```
SELECT
    name, UPPER(name) AS upper_name
FROM Student1;
```

Output:

name	upper_name
Deepu	DEEPU

name	upper_name
Nani	NANI
Rani	RANI
Joseph	JOSEPH
Madhu	MADHU

2.LOWER()

It Convert text of column to lowercase.

Syntax:

LOWER(column_name)

Example:

```
SELECT
    name,
    LOWER(name) AS lower_name
FROM Student1;
```

Output:

name	lower_name
Deepu	deepu
Nani	nani
Rani	rani
Joseph	joseph
Madhu	madhu

3.LENGTH()

It Returns number of characters present in the string.

Syntax:

LENGTH(column_name)

Example:

```
SELECT
    name,
    LENGTH(name) AS name_length
FROM Student1;
```

Output:

name	name_length
Deepu	5
Nani	4
Rani	4
Joseph	6
Madhu	5

4.CONCAT()

It Combines multiple strings.

Syntax:

CONCAT(string1, string2, ...)

Example:

SELECT

CONCAT(name, ' from', dept) AS full_info

FROM Student1;

Output:

full_info

Deepu from CSE

Nani from ECE

Rani from ECE

Joseph from CSE

Madhu from CSE

5. SUBSTRING() Function

It is used to return part of string.

Syntax:

SUBSTRING(string, start_position, length)

Example:

SELECT SUBSTRING(name, 1, 3) AS short_name

FROM Student1;

Output:

name	short_name
Deepu	Dee
Nani	Nan
Rani	Ran
Joseph	Jos
Madhu	Mad

5Q.Analyze the use of GROUP BY and HAVING clauses in SQL to group records and filter grouped data.

GROUP BY CLAUSE:

The GROUP BY clause is a SQL command that is used to group rows that have the same values. The GROUP BY clause is used in the SELECT statement .

GROUP BY Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

HAVING Clause :

The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

HAVING applies to summarized group records, whereas WHERE applies to individual records. Only the groups that meet the HAVING criteria will be returned. HAVING requires that a GROUP BY clause is present.

HAVING Syntax:

```
SELECT column-names FROM table-name
WHERE condition GROUP BY column-names
HAVING condition
```

```
create table Student1(id int,name varchar(50),age int,dept varchar(50));
insert into Student1 values(1,"Deepu",19,"CSE"),
(2,"Nani",21,"ECE"),
(3,"Rani",20,"ECE"),
(4,"Joseph",17,"CSE"),
(5,"Madhu",18,"CSE");
select * from Student1;
```

OUTPUT:

id	name	age	dept
1	Deepu	19	CSE
2	Nani	21	ECE
3	Rani	20	ECE
4	Joseph	17	CSE
5	Madhu	18	CSE

Example:

The following SQL statement lists the department who is having students grater than or equal to 2.

```
SELECT dept, COUNT(*) AS total_students
```

```
FROM Student1
```

```
GROUP BY dept
```

```
HAVING COUNT(*) >= 2;□
```

Output:

dept	total_students
CSE	3
ECE	2

6Q. Write SQL queries to retrieve data from two related tables using:**a) INNER JOIN****b) LEFT JOIN****c) RIGHT JOIN**

- Join operation is essentially a cartesian product followed by a selection criterion.
- Join operation denoted by $\bowtie$.
- JOIN operation also allows joining variously related tuples from different relations.
- A SQL Join statement is used to combine data or rows from two or more tables based on a common field between them.

Or

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.

Different Types of Joins

There are mainly four types of joins that you need to understand. They are:

(INNER) JOIN: Returns records that have matching values in both tables

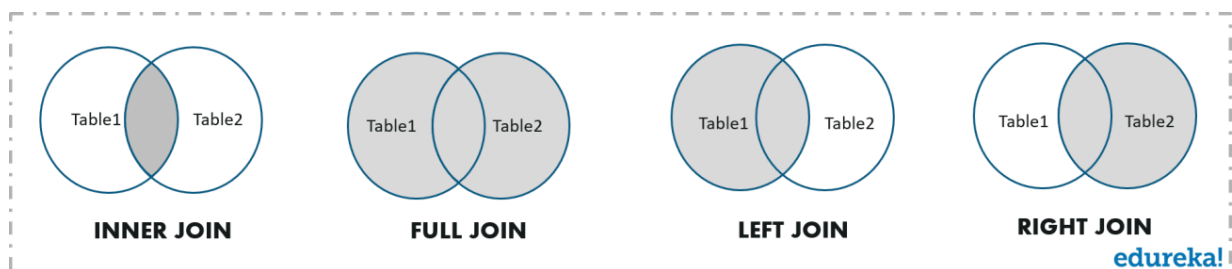
LEFT (OUTER) JOIN: Returns all records from the left table, and the matched records from the right table

RIGHT (OUTER) JOIN: Returns all records from the right table, and the matched records from the left table

FULL (OUTER) JOIN: Returns all records when there is a match in either left or right Table.

Create Tables

```
CREATE TABLE Department (
```



```
DeptID INT PRIMARY KEY,
```

```
DeptName VARCHAR(50) NOT NULL
```

```
);
```

```
CREATE TABLE Student (
```

```
StudentID INT PRIMARY KEY,
```

```
StudentName VARCHAR(50),
```

```
DeptID INT,
```

```
FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
```

```
);
```

```
INSERT INTO Department VALUES
```

```
(1, 'Computer Science'),
```

```
(2, 'Mechanical'),
```

```
(3, 'Civil');
```

```
INSERT INTO Student VALUES
```

```
(101, 'Rahul', 1),
```

```
(102, 'Anita', 2),
```

```
(103, 'Kiran', 1),
```

```
(104, 'Meena', NULL);
```

OUTPUT:

Department Table:

DeptID	DeptName
1	Computer Science
2	Mechanical
3	Civil

Student Table:

StudentID	StudentName	DeptID
101	Rahul	1
102	Anita	2
103	Kiran	1
104	Meena	NULL

OBSERVATIONS:

- Dept 3 has **no students**
- Meena has **no department**

INNER JOIN

The INNER JOIN keyword selects all rows from both the tables as long as the condition satisfies. i.e value of the common field will be same.

INNER JOIN Syntax:

SELECT table1.column1,table1.column2,table2.column1,....

FROM table1

INNER JOIN table2

ON table1.matching_column = table2.matching_column;

Query:

SELECT s.StudentName, d.DeptName

FROM Student s

INNER JOIN Department d

ON s.DeptID = d.DeptID;

Output (Only Matching Records)

StudentName	DeptName
Rahul	Computer Science
Anita	Mechanical
Kiran	Computer Science

- Meena is excluded (no DeptID) and Civil department is also excluded (no students)

LEFT OUTER JOIN

This join returns all the rows of the table on the left side of the join and matching rows for the table on the right side of join. The rows for which there is no matching row on right side, the result-set will contain null.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....  
FROM table1  
LEFT JOIN table2  
ON table1.matching_column = table2.matching_column;
```

Query:

```
SELECT s.StudentName, d.DeptName  
FROM Student s  
LEFT JOIN Department d  
ON s.DeptID = d.DeptID;
```

Output (All Students + Matching Departments)

StudentName	DeptName
Rahul	Computer Science
Anita	Mechanical
Kiran	Computer Science
Meena	NULL

- Shows all students
- Meena appears with NULL department

RIGHT OUTER JOIN

RIGHT JOIN is similar to LEFT JOIN. This join returns all the rows of the table on the right side of the join and matching rows for the table on the left side of join. The rows for which there is no matching row on left side, the result-set will contain *null*.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....  
FROM table1  
RIGHT JOIN table2
```

ON table1.matching_column = table2.matching_column**n;**

Query:

```
SELECT s.StudentName, d.DeptName  
FROM Student s  
RIGHT JOIN Department d  
ON s.DeptID = d.DeptID;
```

Output (All Departments + Matching Students)

StudentName	DeptName
Rahul	Computer Science
Kiran	Computer Science
Anita	Mechanical
NULL	Civil

Observations:

- Civil department appears.
- NULL because no student belongs to Civil.

UNIT 4

SHORT ANSWER QUESTIONS

1Q.What is Schema Refinement.

Schema design is the process of creating a logical and organized structure for a database, which involves defining tables, columns, relationships, constraints, and other elements that will govern how data is stored and accessed. Effective schema design is important for creating a robust, scalable, and efficient database system.

2Q.Define Functional Dependency.

The attributes of a table are said to be dependent on each other when an attribute of a table uniquely identifies another attribute of the same table. A functional dependency is denoted by an arrow $\rightarrow$. The functional dependency of X on Y is represented by $X \rightarrow Y$.

3Q.Define Denormalization.

Denormalization is a database optimization technique where redundant data is intentionally added to one or more tables to reduce the need for complex joins and improve query performance. It is not the opposite of normalization, but rather an optimization applied after normalization.

4Q.Write about BCNF.

Boyce Codd normal form (BCNF)

- BCNF is the advance version of 3NF. It is stricter than 3NF.
- A table is in BCNF if every functional dependency $X \rightarrow Y$, X is the super key of the table.

For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

Example:

Consider a table Student (Student_ID, Course, Instructor)

Student_ID	Course	Instructor
101	DBMS	Ravi
102	OS	Kumar
103	DBMS	Ravi

Functional Dependency:

Course $\rightarrow$ Instructor

Here Course determines Instructor, but Course is not a super key, so the table violates BCNF.

To convert into BCNF, split into two tables:

Course_Instructor (Course, Instructor)

Student_Course (Student_ID, Course)

Now every determinant is a **super key**, so the tables satisfy **BCNF**.

5Q. Write about 4th normal form(4NF).

- A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
- For a dependency $A \twoheadrightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Example:

Consider a table:

Student (Student_ID, Course, Hobby)

Student_ID	Course	Hobby
101	DBMS	Music
101	DBMS	Cricket

Here $\text{Student_ID} \twoheadrightarrow \text{Hobby}$ and $\text{Student_ID} \twoheadrightarrow \text{Course}$ are multivalued dependencies.

To convert into 4NF, split into two tables:

Student_Course (Student_ID, Course)

Student_Hobby (Student_ID, Hobby)

This removes redundancy and satisfies 4NF.

6Q. Write about 5th normal form(5NF).

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.

Example:

Consider a table Supplier_Part_Project (Supplier, Part, Project)

Supplier	Part	Project
S1	P1	J1
S1	P1	J2

To achieve 5NF, split into three tables:

- Supplier_Part (Supplier, Part)
- Supplier_Project (Supplier, Project)
- Part_Project (Part, Project)

By joining these tables, we can get the original table without redundancy. This satisfies 5NF.

LONG ANSWER QUESTIONS

1Q: Describe database anomalies and explain the different types of anomalies that occur in poorly designed database tables.

Anomaly (in DBMS):

An anomaly is a problem that occurs in a database due to data redundancy or poor table design, which may lead to inconsistency or data errors.

Types of Anomalies:

1. Insertion Anomaly
2. Update Anomaly
3. Deletion Anomaly

Here all the data is stored in a single table which causes redundancy of data or say anomalies as SID and Sname are repeated once for same CID . Let us discuss anomalies one by one.

Due to redundancy of data we may get the following problems, those are-

1.insertion anomalies : It may not be possible to store some information unless some other information is stored as well.

InsertionAnomalies: New course is introduced C4, But no student is there who is having C4 subject.

SID	Sname	CID	Cname	FEE
S1	A	C1	C	5k
S2	A	C1	C	5k
S1	A	C2	C	10k
S3	B	C2	C	10k
S3	B	C2	JAVA	15k

NULL	NULL	CA	DB	12k
------	------	----	----	-----

To Insert that Row, It is Required to Put Dummy Data..

Therefore,

xx	xx	CA	DB	12k
----	----	----	----	-----

2.update anomalies: If one copy of redundant data is updated, then inconsistency is created unless all redundant copies of data are updated.

updation anomaly – If there is updation in the fee from 5000 to 7000, then we have to update FEE column in all the rows, else data will become inconsistent.

SID	Sname	CID	Cname	FEE
S1	A	C1	C	5k
S2	A	C1	C	5k
S1	A	C2	C	10k
S3	B	C2	C	10k
S3	B	C2	JAVA	15k

7k > 7k
Costly Operation
↓
More IO Cost

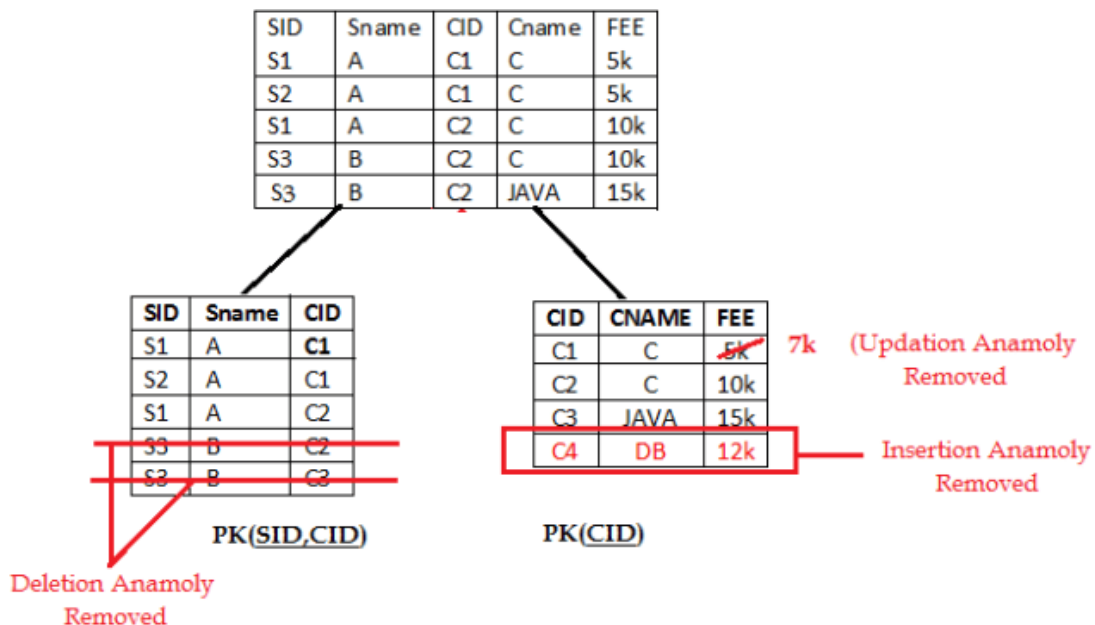
3.deletion anomalies: It may not be possible to delete some information without losing some other information as well.

Deletion of S3 student cause the deletion of course.

SID	Sname	CID	Cname	FEE
S1	A	C1	C	5k
S2	A	C1	C	5k
S1	A	C2	C	10k
S3	B	C2	C	10k
S3	B	C2	JAVA	15k

Because of deletion of some data forced to delete some other useful data.
Insertion Anomaly and Deletion Anomaly- These anomalies exist only due to redundancy, otherwise they do not exist.

Solutions To Anomalies : Decomposition of Tables – Schema Refinement



2Q.Explain the concept of functional dependency and analyze its different types with appropriate examples

Functional dependency :

The attributes of a table are said to be dependent on each other when an attribute of a table uniquely identifies another attribute of the same table. A functional dependency is denoted by an arrow $\rightarrow$. The functional dependency of X on Y is represented by $X \rightarrow Y$.

Example:			
Employee number	Employee Name	Salary	City
1	Dana	50000	San Francisco
2	Francis	38000	London
3	Andrew	25000	Tokyo

In this example, if we know the value of Employee number, we can obtain Employee Name, city, salary, etc. By this, we can say that the city, Employee Name, and salary are functionally depended on Employee number.

Types of Functional Dependencies

- i. Multivalued dependency:
- ii. Trivial functional dependency:
- iii. Non-trivial functional dependency:
- iv. Transitive dependency:

i) Multivalued dependency in DBMS

Multivalued dependency occurs in the situation where there are multiple independent multivalued attributes in a single table. A multivalued dependency is a complete constraint between two sets of attributes in a relation. It requires that certain tuples be present in a relation.

Example:

Car_model	Maf_year	Color
H001	2017	Metallic
H001	2017	Green
H005	2018	Metallic
H005	2018	Blue
H010	2015	Metallic
H033	2012	Gray

In this example, maf_year and color are independent of each other but dependent on car_model. In this example, these two columns are said to be multivalued dependent on car_model.

This dependence can be represented like this:

car_model -> maf_year car_model -> colour

ii) Trivial Functional dependency:

The Trivial dependency is a set of attributes which are called a trivial if the set of attributes are included in that attribute.

So, $X \rightarrow Y$ is a trivial functional dependency if Y is a subset of X .

For example:

Emp_id	Emp_name
--------	----------

AS555	Harry
-------	-------

AS811	George
-------	--------

AS999	Kevin
-------	-------

Consider this table with two columns Emp_id and Emp_name.

$\{Emp_id, Emp_name\} \rightarrow Emp_id$ is a trivial functional dependency as

Emp_id is a subset of

$\{Emp_id, Emp_name\}$.

iii) Non trivial functional dependency:

Functional dependency which also known as a nontrivial dependency occurs when $A \rightarrow B$

holds true where B is not a subset of A . In a relationship, if attribute B is not a subset of attribute A , then it is considered as a non-trivial dependency.

Example:

Company	CEO	Age
Microsoft	Satya Nadella	51
Google	Sundar Pichai	46
Apple	Tim Cook	57

{Company} -> {CEO} (if we know the Company, we know the CEO name)

But CEO is not a subset of Company, and hence it's non-trivial functional dependency.

iv) Transitive dependency:

A transitive is a type of functional dependency which happens when it is indirectly formed by two functional dependencies.

Example:		
Company	CEO	Age
Microsoft	Satya Nadella	51
Google	Sundar Pichai	46
Alibaba	Jack Ma	54

{Company} -> {CEO} (if we know the company, we know its CEO's name)

{CEO} -> {Age} If we know the CEO, we know the Age Therefore according to the rule of transitive dependency:

{Company} -> {Age} should hold, that makes sense because if we know the company name, we can know his age.

3Q.Explain the concept of normalization in DBMS and apply the rules of 1NF, 2NF, and 3NF to organize data in a database table.

Normalization

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate the undesirable characteristics like Insertion, Update and Deletion Anomalies.
- Normalization divides the larger table into the smaller table and links them using relationship.
- The normal form is used to reduce redundancy from the database table.

First normal form(1NF):

A relation is said to be in first normal form if it contains all atomic values or single values.

Example:

Domain	Courses
Programming	C , java
Web designing	HTML , PHP

The above table consist of multiple values in single columns which can be reduced into atomic values by using first normal form as follows-

Domain	Courses
Programming	C
Programming	Java
Web designing	HTML
Web designing	PHP

Second normal form(2NF):

A relation is said to be in second normal form if it is in first normal form without any partial dependencies.

☐ In second normal form non-prime attributes should not depend on proper subset of key attributes.

Example:

StudentID	StudentName	ProjectID	ProjectName
S101	Ravi	P01	Library System
S101	Ravi	P02	Attendance System
S102	Anitha	P01	Library System

Here (student id, project id) are key attributes and (student name, project name) are non-prime attributes. It is decomposed as-

StudentID (PK)	StudentName	ProjectID
S101	Ravi	P01
S102	Anitha	P02
S103	Kumar	P01

ProjectID	ProjectName
P01	Library System
P02	Attendance System

Third normal form(3NF): A relation is said to be in third normal form , if it is already in second normal form and no transitive dependencies exists.

Transitive dependency – If $A \rightarrow B$ and $B \rightarrow C$ are two FDs then $A \rightarrow C$ is called transitive dependency.

A relation is in 3NF if at least one of the following condition holds in every non-trivial function dependency $X \rightarrow Y$

X is a super key.

Y is a prime attribute (each element of Y is part of some candidate key).

StudentID	StudentName	City	Country	ZIP
S101	Ravi	Hyderabad	India	500001
S102	Anitha	Vijayawada	India	520001
S103	Kumar	Hyderabad	India	500001

It is decomposed as:

Table 1:Student

StudentID (PK)	StudentName	City (FK)
S101	Ravi	Hyderabad
S102	Anitha	Vijayawada
S103	Kumar	Hyderabad

Table 2:City

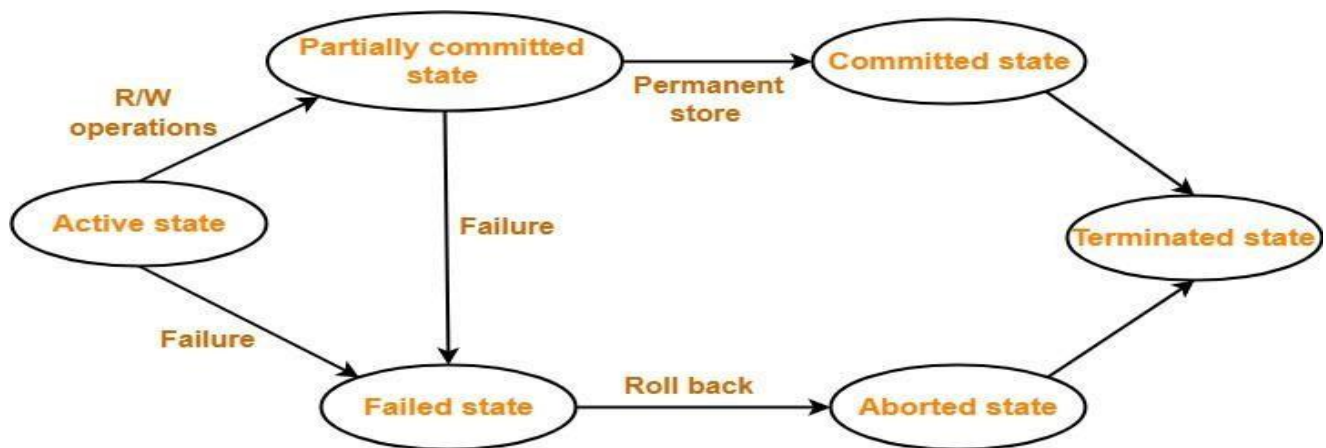
City (PK)	Country	ZIP
Hyderabad	India	500001
Vijayawada	India	520001

4Q. Explain the concept of transaction management in DBMS and briefly explain about transaction states.

Transaction Management in DBMS:

Transaction Management in a Database Management System (DBMS) is used to control and manage database transactions so that the database remains accurate, consistent, and reliable even if errors or system failures occur.

A transaction is a sequence of database operations such as insert, update, delete, or retrieve, performed as a single logical unit of work. A transaction must either complete fully or not execute at all.



Transaction States in DBMS

Transaction States-

A transaction goes through many different states

throughout its life cycle. These states are called as

transaction states. Transaction states are as follows-

- i. Active state
- ii. Partially committed state
- iii. Committed state
- iv. Failed state
- v. Aborted state
- vi. Terminated state

i) **Active State-**

This is the first state in the life cycle of a transaction.

- A transaction is called in an **active state** as long as its instructions are getting executed.
- All the changes made by the transaction now are stored in the buffer in main memory.

ii) **Partially Committed State-**

After the last instruction of transaction has executed, it enters into a **partially committed state**.

- After entering this state, the transaction is considered to be partially committed.
- It is not considered fully committed because all the changes made by the transaction are still stored in the buffer in main memory.

iii) **Committed State-**

- After all the changes made by the transaction have been successfully stored into the database, it enters into a **committed state**.
- Now, the transaction is considered to be fully committed.

NOTE-

- After a transaction has entered the committed state, it is not possible to roll back the transaction.
- In other words, it is not possible to undo the changes that has been made by the transaction.
- This is because the system is updated into a new consistent state.
- The only way to undo the changes is by carrying out another transaction called as **compensating transaction** that performs the reverse operations.

iv) **Failed State-**

When a transaction is getting executed in the active state or partially committed state and some failure occurs due to which it becomes impossible to continue the execution, it enters into a failed state.

v) **Aborted State-**

After the transaction has failed and entered into a failed state, all the changes made by it have to be undone.

- To undo the changes made by the transaction, it becomes necessary to roll back the transaction.
- After the transaction has rolled back completely, it enters into an **aborted state**.

vi) Terminated State-

- This is the last state in the life cycle of a transaction.
- After entering the committed state or aborted state, the transaction finally enters into a **terminated state** where its life cycle finally comes to an end.

5Q. Discuss the ACID properties of transactions and analyze their importance in maintaining database reliability and consistency.

ACID Properties

- It is important to ensure that the database remains consistent before and after the transaction.
- To ensure the consistency of database, certain properties are followed by all the transactions occurring in the system.
- These properties are called as **ACID Properties** of a transaction.

A	= Atomicity
C	= Consistency
I	= Isolation
D	= Durability

1) Atomicity-

- This property ensures that either the transaction occurs completely or it does not occur at all.
- In other words, it ensures that no transaction occurs partially.
- That is why, it is also referred to as —**All or nothing rule**It is the responsibility of Transaction Control Manager to ensure atomicity of the transactions.

II) Consistency-

- This property ensures that integrity constraints are maintained.
- In other words, it ensures that the database remains consistent before and after the transaction.
- It is the responsibility of DBMS and application programmer to ensure consistency of the database.

III) Isolation-

- This property ensures that multiple transactions can occur simultaneously without causing any inconsistency.
- During execution, each transaction feels as if it is getting executed alone in the system.
- A transaction does not realize that there are other transactions as well getting executed parallelly.
- Changes made by a transaction becomes visible to other transactions only after they are written in the memory.
- The resultant state of the system after executing all the transactions is same as the state that would be achieved if the transactions were executed serially one after the other.
- It is the responsibility of concurrency control manager to ensure isolation for all the transactions.

IV) Durability-

- This property ensures that all the changes made by a transaction after its successful execution are written successfully to the disk.
- It also ensures that these changes exist permanently and are never lost even if there occurs a failure of any kind.
- It is the responsibility of recovery manager to ensure durability in the database.

6Q.Describe the concept of concurrency control and illustrate how it helps manage simultaneous transactions with a suitable example.

Concurrency execution

Concurrency execution in a Database Management System (DBMS) involves the interleaved or simultaneous operation of multiple transactions to enhance performance, while key concepts and protocols ensure the data remains correct and reliable. This is critical for systems with many simultaneous users, like online banking or e-commerce platforms.

Key Concepts in Concurrency Control

The goal of concurrency control is to maintain the database's integrity despite overlapping

operations. This is framed by the **ACID properties**:

- **Atomicity:** Transactions are "all or nothing." If a failure occurs, the entire transaction

is

undone (rolled back).

- **Consistency:** Each transaction must move the database from one valid state to another.

- **Isolation:** The execution of one transaction must be isolated from others. The result of concurrent execution must be the same as if the transactions ran sequentially (**serializability**).

- **Durability:** Committed changes are permanent and survive system crashes.

Concurrency Execution Example: The "Unrepeatable Read" Problem

2223

The "Unrepeatable Read" scenario occurs when a transaction reads the same data item twice but gets different values because another committed transaction modified the item in between the two reads.

Scenario:

- A user wants to check the current stock count for an item (e.g., 50 units) and then confirm an order that depends on that count.

Execution without Strict Isolation (e.g., in a weak isolation level):

Time	Transaction T1 (Check stock & Order)	Transaction T2 (Stock Restock)	Database State (Stock Count)
T1	Read Stock Count -> 50		50
T2	(User is reviewing the item details)		50
T3		Read Stock Count -> 50	50
T4		Add 20 to Stock Count: $50 + 20 = 70$	50
T5		Write Stock Count -> 70	70
T6		Commit T2	70
T7	Read Stock Count again -> 70		70
T8	(T1 now sees a different value than the first read)		70

Problem: T1 sees a conflict. The first time it checked the stock, it was 50; the second time, it was 70. This unrepeatable read can lead to logic errors within T1—the initial condition T1 evaluated is no longer true when it attempts to complete its operation.

Concurrency Control Mechanism: Strict Two-Phase Locking (2PL)

To prevent unrepeatable reads, DBMSs implement stricter **isolation levels**, typically using locking mechanisms.

Shared Locks (S-locks): Multiple transactions can hold a shared lock simultaneously (for reading data).

Exclusive Locks (X-locks): Only one transaction can hold an exclusive lock at a time (for writing data).

In the example above, if T1 uses a strict isolation level that prevents unrepeatable reads, T1 would place a **shared lock** on the stock count at T1. When T2 attempts to place an **exclusive lock** to modify the stock at T4, T2 is blocked until T1 completes its entire operation and releases its shared lock, thus guaranteeing that T1 always reads the same consistent value within its duration.

UNIT 5

SHORT ANSWER QUESTIONS

1Q.What is a NoSQL Database? Explain its main features.

NoSQL Database

A NoSQL database is a non-relational database that is designed to store and manage large amounts of unstructured or semi-structured data.

Unlike traditional relational databases, NoSQL databases do not use tables and fixed schemas.

Main Features:

- Schema-less design
- High scalability
- High performance for large data
- Supports distributed databases
- Stores data as documents, key-value pairs, graphs, or columns

Example: MongoDB, Cassandra, Redis

2Q.Differentiate between Collections in MongoDB and Tables in SQL Databases.

Feature	Collection (MongoDB)	Table (SQL)
Structure	Stores documents	Stores rows and columns
Schema	Schema flexible	Fixed schema
Data format	JSON / BSON documents	Structured rows
Modification	Easy to change structure	Hard to change structure

Thus, a **Collection in MongoDB is similar to a Table in SQL but without fixed schema.**

3Q.What is a Document in MongoDB? How is it different from a Record in Relational Databases?

Document in MongoDB

A document is a basic unit of data in MongoDB stored in **BSON (Binary JSON) format**.

Example:

```
{  
  name: "Ravi",  
  course: "DBMS",  
  marks: 85  
}
```

Difference between Document and Record

Document (MongoDB)	Record (SQL)
Stored in BSON format	Stored as row
Flexible structure	Fixed structure
Can contain nested objects	Cannot store nested objects easily

4Q.What is Schema Flexibility in NoSQL Databases?

Schema flexibility means that a NoSQL database does not require a fixed structure for storing data.

Different documents in the same collection can have different fields.

Example:

```
{ name: "Ravi", age: 20 }  
  
{ name: "Sita", course: "DBMS", marks: 90 }
```

This flexibility allows developers to easily modify the database structure without altering the entire database.

5Q.What are CRUD operations in MongoDB?

CRUD operations are the basic operations performed on database

data. CRUD stands for: **C – Create** → Insert data **R – Read** →

Retrieve data **U – Update** → Modify data **D – Delete** → Remove data

Example:**Insert:**

```
db.students.insertOne({name:"Ravi",marks:90})
```

Read:

```
db.students.find()
```

Update:

```
db.students.updateOne({name:"Ravi"},{$set:{marks:95}})
```

Delete:

```
db.students.deleteOne({name:"Ravi"})
```

6Q.What is Projection in MongoDB Queries?

Projection is used to select specific fields from documents in a collection. Instead of retrieving all fields, projection allows retrieving only required fields. **Example:**

```
db.students.find({}, {name:1, marks:1})
```

This query displays only **name and marks fields**.

Projection improves **query performance and reduces data transfer**.

LONG ANSWER QUESTIONS

1Q.Explain the architecture and concepts of NoSQL databases with suitable examples.

NoSQL Databases

NoSQL databases are non-relational databases designed to handle large volumes of unstructured or semi-structured data.

They are widely used in **big data and real-time web applications**. Example: MongoDB, Cassandra, Redis.

Architecture of NoSQL

NoSQL databases follow a **distributed architecture** where data is stored across multiple servers.

Features of architecture:

- Distributed data storage
- Horizontal scalability
- Fault tolerance
- High availability

Types of NoSQL Databases

1. Key-Value Databases

Data stored as key-value

pairs. **Example:**

```
Key : 101  
Value : Ravi
```

Example systems: Redis, DynamoDB

2. Document Databases

Data stored as documents in JSON format.

Example (MongoDB):

```
{  
  name: "Ravi",  
  course: "Python"  
}
```

3. Column-based Databases

Data stored in columns instead of rows.

Example: Cassandra

4. Graph Databases

Used for relationship-based data.

Example: Neo4j

2Q. Describe how to set up MongoDB and create a database and collection.

Steps to Setup MongoDB

1. Download MongoDB from official website.
2. Install MongoDB on the system.
3. Open MongoDB shell or MongoDB Compass.

Creating a Database

use student_db

If the database does not exist, MongoDB will create it automatically.

Creating a Collection

```
db.createCollection("students")
```

Example document insertion:

```
db.students.insertOne(  
{  
  name:"Ravi",  
  course:"DBMS",  
  marks:85  
})
```

Viewing Data

```
db.students.find()
```

3Q.Explain CRUD operations in MongoDB with examples.

CRUD operations are basic operations used to manage data in MongoDB.

1. Create Operation

Used to insert new documents into a collection.

Example:

```
db.students.insertOne(  
{  
  name:"Ravi",  
  course:"Python",  
  marks:85  
})
```

2. Read Operation

Used to retrieve data from the database.

Example:

```
db.students.find()
```

Retrieve specific data:

```
db.students.find({course:"Python"})
```

3. Update Operation

Used to modify existing documents.

Example:

```
db.students.updateOne(  
  {name:"Ravi"},  
  {$set:{marks:90}}  
)
```

4. Delete Operation

Used to remove documents from the collection.

Example:

```
db.students.deleteOne({name:"Ravi"})
```

4Q.Explain MongoDB query operators including Conditional Operators and Logical Operators with examples

Conditional Operators Used to compare values.MongoDB provides different operators to perform queries.

1) Common operators:

Operator	Meaning
\$gt	Greater than
\$lt	Less than
\$gte	Greater than or equal
\$lte	Less than or equal
\$ne	Not equal

Example:

```
db.students.find({marks:{$gt:80}})
```

2) Logical Operators

Used to combine multiple

conditions. Common operators:

Operator	Meaning
\$and	Logical AND
\$or	Logical OR
\$not	Logical NOT

Example:

```
db.students.find({
  $and:[
    {marks:{$gt:80}},
    {course:"DBMS"}
  ]
})
```

5Q.Explain Nested Documents and Array Fields in MongoDB with suitable examples.**Nested Documents**

Nested documents store a document inside another document.

Example:

```
{
  name:"Ravi",
  address:{
    city:"Hyderabad",
    pincode:500001
  }
}
```

Access nested field:

```
db.students.find({"address.city":"Hyderabad"})
```

Array Fields

MongoDB allows storing multiple values in an array.

Example:

```
{
  name:"Ravi",
  skills:["Python","Java","MongoDB"]
}
```

Query array field:

```
db.students.find({skills:"Python"})
```

6Q.Compare SQL and NoSQL Databases.

SQL Databases (Relational Databases) and **NoSQL Databases (Non-Relational Databases)** are two different types of database systems used for storing and managing data.

SQL databases use structured tables and fixed schemas, while NoSQL databases are designed to handle large volumes of unstructured or semi-structured data with flexible schemas.

SQL databases are commonly used in traditional applications such as banking systems, enterprise applications, and transaction processing systems.

NoSQL databases are widely used in modern applications such as big data systems, social media platforms, and real-time web applications.

Comparison between SQL and NoSQL Databases

Feature	SQL Database	NoSQL Database
Database Type	Relational Database	Non-Relational Database
Structure	Data stored in tables (rows and columns)	Data stored as documents, key- value pairs, graphs, or columns
Schema	Fixed schema (structure must be defined before storing data)	Flexible or schema-less
Scalability	Vertical scaling (increase hardware power)	Horizontal scaling (add more servers)
Data Type	Mainly structured data	Structured, semi-structured, and unstructured data
Query Language	Uses Structured Query Language (SQL)	Uses different query methods depending on database
Performance	Suitable for complex queries and transactions	Suitable for large scale distributed systems
Examples	MySQL, Oracle, SQL Server	MongoDB, Cassandra, Redis